به نام خدا

دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

درس طراحی الگوریتم پیشرفته

تمرین دستی اول

نام و نام خانوادگی	بهزاد جنتی
شماره دانشجویی	810103098
تاریخ ارسال تمرین	1403.12.17
نام استاد	دکتر هشام فیلی

فهرست

سوال1 – کاربردهای طراحی الگوریتم 2

سوال 2 _ مشکلات پردازش دکتر کوشا پردازشی.....9

سوال 3 _ دکتر کوشا پردازشی محقق موفق.....17

سوال 4 - وزیر نیرو vs دکتر کوشا پردازشی.....25

سوال 5 _ فرزانگان دربار vs دیوها.....28

سوال 6 _ سپر حفاظتی تونی استارک.....32

سوال 1 – کاربردهای طراحی الگوریتم

الف) تبدیل فوریه سریع

الف) تبدیل فوریه سریع یک الگوریتم برای محاسبه ی فوریه یک سیگنال با سرعت بالا می باشد. توضیح دهید این الگوریتم چگونه از روش تقسیم و غلبه برای محاسبه فوریه یک سیگنال با سرعت بالا استفاده میکند.

در این سوال میخواهیم به محاسبه‌ی تبدیل فوریه سریع (Fast Fourier Transform یا FFT) و توضیح چگونگی استفاده از روش تقسیم و غلبه در این الگوریتم بپردازیم.

تبدیل فوریه(Fourier Transform):

تبدیل فوریه یک ابزار ریاضی است که برای تجزیه یک سیگنال یا تابع به فرکانس‌های تشکیل‌دهنده آن استفاده می‌شود. این تبدیل در حوزه‌های مختلف مثل پردازش سیگنال، تصویر و صوت کاربرد دارد.

فرمول تبدیل فوریه گسسته (Discrete Fourier Transform) یا DFT به صورت زیر تعریف می‌شود:

$$X(k) = \sum_{n=0}^{n-1} x(n).e^{-j\frac{2\pi kn}{N}}, \qquad k = 0,1,2,\dots,N-1$$

که در این فرمول:

- x(n) نمونه های سیگنال در زمان هستند.
- X(k) ضرایب تبدیل فوریه در حوزه فرکانس هستند.
- N تعداد نقاط نمونه برداری شده است.

مشکل اصلی DFT این است که برای پیاده سازی مستقیم آن (محاسبه مستقیم فرمول) به $O(N^2)$ عملیات محاسباتی داریم که برای سیگنالهای بزرگ بسیار کند است.

تبدیل فوریه سریع(FFT):

تبدیل فوریه سریع (Fast Fourier Transform یا FFT): یک الگوریتم کارآمد برای محاسبه تبدیل فوریه گسسته (DFT) است که از روش تقسیم و غلبه برای افزایش سرعت محاسبات استفاده می‌کند.

هدف: کاهش پیچیدگی محاسباتی از $O(N^2)$ به $O(NlogN)$ با استفاده از روش تقسیم و غلبه

توضیح روش تقسیم و غلبه در FFT:

1. تقسیم: سیگنال اصلی با اندازه N به دو زیر سیگنال کوچکتر تقسیم می‌شوند:

- ✓ یک زیر سیگنال شامل نمونه های زوج (x[2n]) و دیگری شامل نمونه های فرد (x[2n+1])

2.غلبه: برای هر یک از این زیر سیگنالها (محاسبه DFT برای نمونه های فرد و زوج)، تبدیل فوریه به صورت جداگانه محاسبه میشود.

سپس نتایج ترکیب میشوند تا تبدیل فوریه کل سیگنال بدست بیاید.

روش کار الگوریتم FFT با روش تقسیم و غلبه(Cooley-Tukey):

مراحل:

1. ورودی: سیگنال x(n) با تعداد نمونه ها N ، که فرض میشود N یک توان از 2 است(یعنی $N = 2^m$)

2. تقسیم سیگنال: سیگنال x(n) را به دو بخش (زیر سیگنال) تقسیم میکنیم:

$$x_{\text{زوج}}[n] = x[2n]$$

$$x_{\text{فرد}}[n] = x[2n+1]$$

3. محاسبه تبدیل فوریه زیر سیگنالها: سپس DFT هر بخش را حساب میکنیم(تبدیل فوریه روی هر دو زیر سیگنال (x[2n],x[2n+1]) به صورت بازگشتی اعمال میشود.)

$$X_{\text{فرد}}(k) = DFT\{x_{\text{فرد}}[n]\} \quad ، \quad X_{\text{زوج}}(k) = DFT\{x_{\text{زوج}}[n]\}$$

4. ترکیب نتایج: از فرمولهای زیر برای ترکیب نتایج استفاده میشود:

برای k = 0, 1, 2,…,N/2-1 :

$$X(k) = X_{\text{زوج}}(k) + e^{-j\frac{2\pi k}{N}}.X_{\text{فرد}}(\mathrm{k})$$

برای k = N/2, N/2+1, …, N-1 :

$$X(k) = X_{\text{زوج}}\left(k - \frac{N}{2}\right) - e^{-j\frac{2\pi(k-N/2)}{N}}.X_{\text{فرد}}\left(\mathrm{k} - \frac{\mathrm{N}}{2}\right)$$

نکته) در فرمول دوم میتوانستیم طرفین تساوی را با یک N/2 جمع کنیم(داخل پرانتزها با k) که فرم دوم همین فرمول بدست میاد. هم چنین در برخی از کتابهای مرجع از $W_N^k = e^{-j\frac{2\pi k}{N}}$ به عنوان عوامل چرخشی (twiddle factors) یاد میشود.

5. بازگشتی : این فرآیند تقسیم و ترکیب به صورت بازگشتی تا رسیدن به DFT های با طول 1 ادامه می‌یابد.

6. خروجی: مقادیر X(k) (شامل ضرایب فوریه در حوزه فرکانس) برای k = 0, 1,…,N-1

ویژگیهای مهم FFT:

- پیچیدگی محاسباتی:

الگوریتم FFT با استفاده از روش تقسیم و غلبه پیچیدگی را از $O(N^2)$ به $O(NlogN)$ کاهش میدهد.

$$T(N) = 2T(N/2) + O(N) \rightarrow T(N) = O(N \log N)$$

  - ✓ در هر مرحله، مسئله به دو زیرمسئله با اندازه N/2 تقسیم می‌شود
  - ✓ تعداد مراحل log N است
  - ✓ در هر مرحله، O(N) عملیات انجام می‌شود

- بازگشتی بودن:

الگوریتم به صورت بازگشتی عمل میکند و تا زمانی که طول سیگنال به 1 برسد، سیگنال را تقسیم میکند.

- تقارن و بازگشت:

از تقارن و خاصیت های ریاضی W_N^k (عوامل چرخشی) برای کاهش محاسبات اضافی استفاده میشود.

ب) حل مشکل پیش بینی دوستی های آینده در شبکه های اجتماعی با استفاده از الگوریتم تقسیم و غلبه

ب) یک شبکه اجتماعی را می‌توان به‌صورت یک گراف نمایش داد که در آن:

- کاربران به‌عنوان گره‌ها هستند.
- روابط دوستی به‌عنوان یال‌های بین گره‌ها نمایش داده می‌شوند.

یک از وظایف در شبکه های اجتماعی این است که دوستی‌های آینده بر اساس ارتباطات فعلی بین کاربران پیش بینی شود. یک روش ساده اما کند برای این کار، استفاده از الگوریتم‌های مبتنی بر مسیر مانند کاتز (Katz) یا پیج‌رنک (PageRank) است. با این حال، این روش‌ها برای گراف‌های بزرگ بسیار پرهزینه و کند هستند. برای حل این مشکل از الگوریتم تقسیم و غلبه کمک میگیرند. راه حل ارایه شده را بررسی کنید و توضیح دهید.

در این مسئله با یک شبکه اجتماعی مواجه هستیم که به صورت گراف نمایش داده می‌شود:

- گره ها نشان دهنده کاربران هستند.
- یال ها نشان دهنده روابط دوستی بین کاربران هستند.

هدف: پیش بینی دوستی های آینده بر اساس ارتباطات فعلی بین کاربران

روش‌های مبتنی بر مسیر مانند الگوریتم کاتز (Katz) یا پیج‌رنک (PageRank) برای این کار مناسب هستند، اما برای گراف‌های بزرگ بسیار پرهزینه و کند می‌شوند. به همین دلیل از الگوریتم تقسیم و غلبه استفاده می‌کنیم.

راه حل مبتنی بر تقسیم و غلبه:

اصول کلی الگوریتم تقسیم و غلبه برای این مسئله:

الگوریتم تقسیم و غلبه شامل سه مرحله اصلی است:

تقسیم: مسئله را به زیرمسائل کوچکتر تقسیم می‌کنیم.

حل: زیرمسائل را به صورت مستقل حل می‌کنیم.

ترکیب: نتایج زیرمسائل را برای حل مسئله اصلی ترکیب می‌کنیم.

پیاده سازی الگوریتم برای پیش بینی دوستی:

مرحله تقسیم:

1. گراف شبکه اجتماعی را به چندین زیرگراف تقسیم می‌کنیم.

می‌توان از روش‌های خوشه‌بندی گراف مانند الگوریتم Louvain یا Kernighan-Lin استفاده کرد.

هدف: ایجاد زیرگراف‌هایی که درون آن‌ها تراکم یال بیشتر و بین آن‌ها تراکم یال کمتر باشد.

2. تقسیم بندی می‌تواند بر اساس:

- اجتماعات موجود در شبکه(communities)
- مناطق جغرافیایی
- گروه های سنی
- علایق مشترک
- یا هر ویژگی دیگری که باعث خوشه بندی طبیعی کاربران میشود.

مرحله حل:

1. برای هر زیر گراف، الگوریتم پیش بینی دوستی را به صورت مستقل اجرا میکنیم.

- در این مرحله می‌توان از الگوریتم‌های مبتنی بر مسیر مانند کاتز یا پیج‌رنک استفاده کرد.
- چون زیرگراف‌ها کوچکتر هستند، اجرای این الگوریتم‌ها روی آن‌ها سریع‌تر خواهد بود.

2. برای هر زیرگراف، جفت کاربرانی که احتمال دوستی آینده بین آن‌ها بالاست را شناسایی می‌کنیم.

مرحله ترکیب:

1. نتایج به دست آمده از زیرگراف‌ها را ترکیب می‌کنیم.
2. برای یال‌های بین زیرگراف‌ها (روابط بین اجتماعات مختلف):

- می‌توان از روش‌های سبک‌تر مانند شباهت همسایگی (Common Neighbors) یا ضریب ژاکارد (Jaccard Coefficient) استفاده کرد.
- یا می‌توان یک الگوریتم سطح بالاتر روی گراف خلاصه‌شده (که هر گره آن یک زیرگراف است) اجرا کرد.

3. نتایج را بر اساس امتیاز پیش‌بینی مرتب کرده و دوستی‌های با احتمال بالاتر را پیشنهاد می‌دهیم.

مزایای استفاده از روش تقسیم و غلبه:

کاهش پیچیدگی محاسباتی:

- اگر گراف اصلی N گره داشته باشد و به k زیرگراف تقسیم شود، پیچیدگی از $O(N^3)$ به حدود $O(k\times(N/k)^3) = O(N^3/k^2)$ کاهش می‌یابد.
- این کاهش برای گراف‌های بزرگ بسیار چشمگیر است.

قابلیت پردازش موازی:

- زیرمسائل می‌توانند به صورت موازی پردازش شوند.
- این امر باعث افزایش سرعت محاسبات می‌شود.

مقیاس‌پذیری:

این روش برای شبکه‌های اجتماعی بزرگ با میلیون‌ها کاربر قابل استفاده است.

دقت قابل قبول:

- اگرچه ممکن است دقت کمی نسبت به روش‌های جامع کاهش یابد، اما در عمل نتایج قابل قبولی ارائه می‌دهد.
- بیشتر دوستی‌های آینده بین کاربرانی شکل می‌گیرد که در یک اجتماع مشترک هستند.

چالش‌ها و راه‌حل‌ها:

انتخاب روش تقسیم مناسب:

تقسیم نامناسب می‌تواند باعث از دست رفتن اطلاعات مهم شود.

راه‌حل: استفاده از الگوریتم‌های خوشه‌بندی پیشرفته مانند Infomap یا Spectral Clustering

روابط بین زیرگراف‌ها:

ممکن است روابط مهمی بین زیرگراف‌های مختلف وجود داشته باشد.

راه‌حل: در نظر گرفتن همپوشانی بین زیرگراف‌ها یا ایجاد یک لایه اضافی برای تحلیل روابط بین زیرگراف‌ها.

به‌روزرسانی:

شبکه‌های اجتماعی به طور مداوم در حال تغییر هستند.

راه‌حل: اجرای دوره‌ای الگوریتم یا طراحی نسخه افزایشی (incremental) از الگوریتم

جمع بندی:

الگوریتم های تقسیم و غلبه یک راه حل کارآمد برای پیش بینی دوستی های آینده در شبکه های اجتماعی بزرگ است. این روش با تقسیم گراف به زیر گراف های کوچکتر و حل مسئله به صورت محلی، پیچیدگی محاسباتی را به طور قابل توجهی کاهش می‌دهد و امکان پردازش شبکه‌های اجتماعی با مقیاس بزرگ را فراهم می‌کند. اگرچه ممکن است دقت کمی نسبت به روش‌های جامع کاهش یابد، اما مزایای آن از نظر کارایی و مقیاس‌پذیری، آن را به گزینه‌ای مناسب برای سیستم‌های واقعی تبدیل می‌کند.

سوال 2 (روش 1) – مشکلات پردازش دکتر کوشا پردازشی

الگوریتم پیشنهادی: تقسیم و غلبه برای یافتن نزدیک‌ترین جفت نقاط

برای حل این مسئله، می‌توان از الگوریتم‌های مختلفی استفاده کرد. یکی از بهترین روش‌ها استفاده از الگوریتم‌های تقسیم و غلبه مانند **Divide and Conquer Closest Pair** است که برای یافتن نزدیک‌ترین جفت نقاط در فضای دوبعدی بهینه است.

مراحل الگوریتم:

1. **مرتب‌سازی**: ابتدا نقاط را بر اساس مختصات x مرتب می‌کنیم (با پیچیدگی O(n log n))
2. **تقسیم**: مجموعه نقاط را به دو قسمت تقریباً مساوی تقسیم می‌کنیم
3. **غلبه**: به صورت بازگشتی، نزدیک‌ترین جفت را در هر نیمه پیدا می‌کنیم
4. **ترکیب**: نزدیک‌ترین جفت ممکن که در دو نیمه مختلف قرار دارند را بررسی می‌کنیم

پیچیدگی زمانی:

- مرتب‌سازی اولیه: O(n log n)
- تابع بازگشتی: $T(n) = 2T(n/2) + O(n)$
- پیچیدگی کل: O(n log n)

پیچیدگی فضایی:

- O(n) برای ذخیره نقاط و آرایه‌های کمکی

پیاده‌سازی نهایی:
برای حل مسئله دکتر پردازشی، می‌توانیم الگوریتم را به صورت زیر پیاده‌سازی کنیم:

1. تمام درخواست‌ها را به صورت نقاط در فضای دوبعدی در نظر می‌گیریم
2. با استفاده از الگوریتم تقسیم و غلبه، تمام جفت نقاطی که فاصله اقلیدسی کمتر از 1 دارند را پیدا می‌کنیم
3. از هر جفت نقطه نزدیک، یکی را به لیست رد شده‌ها اضافه می‌کنیم
4. درخواست‌هایی که در لیست رد شده‌ها نیستند را تایید می‌کنیم

این الگوریتم به صورت کارآمد با پیچیدگی O(n log n) تمام درخواست‌های مشابه را شناسایی می‌کند و مسئله دکتر پردازشی را حل می‌کند.

بخش ب: تغییر الگوریتم برای پذیرش تا 3 درخواست مشابه

در بخش ب از ما خواسته شده که الگوریتم را طوری تغییر دهیم که تا وجود سه درخواست مشابه را قبول کند. این یعنی:

1. دکتر پردازشی می‌خواهد اجازه دهد حداکثر سه درخواست از یک منطقه جغرافیایی مشابه وجود داشته باشد.
2. اگر تعداد درخواست‌های مشابه (با فاصله کمتر از 1) از یک منطقه به بیش از سه مورد برسد، آنگاه باید این موضوع را تشخیص دهیم.
3. این کار نیاز به شناسایی خوشه‌های درخواست‌های مشابه و شمارش تعداد درخواست‌ها در هر خوشه دارد.

برای این منظور، باید الگوریتم بخش الف را گسترش دهیم تا بتواند:

1. تمام جفت‌های نقاط با فاصله کمتر از 1 را شناسایی کند
2. این جفت‌ها را به خوشه‌های کامل گروه‌بندی کند
3. تعداد نقاط در هر خوشه را شمارش کند
4. خوشه‌هایی با بیش از 3 نقطه را شناسایی کند

توضیحات تکمیلی:

1. **ساخت گراف همسایگی**: هر درخواست را به عنوان یک رأس در نظر می‌گیریم و بین درخواست‌های مشابه (با فاصله کمتر از 1) یک یال قرار می‌دهیم.
2. **یافتن اجزای همبند**: با استفاده از الگوریتم جستجوی عمق اول (DFS)، تمام اجزای همبند گراف را پیدا می‌کنیم. هر جزء همبند نشان‌دهنده یک خوشه از درخواست‌های مشابه است.
3. **شمارش اندازه خوشه‌ها**: برای هر خوشه، تعداد درخواست‌های موجود در آن را شمارش می‌کنیم و خوشه‌هایی با بیش از 3 درخواست را شناسایی می‌کنیم.

4. **مدیریت درخواست‌های جدید**: می‌توانیم الگوریتم را به صورت افزایشی پیاده‌سازی کنیم تا با دریافت هر درخواست جدید، آن را با درخواست‌های موجود مقایسه کنیم و تصمیم بگیریم که آیا قابل قبول است یا خیر:

مزایای این رویکرد:

1. **انعطاف‌پذیری**: می‌توانیم به راحتی حداکثر تعداد مجاز درخواست‌های مشابه را تغییر دهیم.
2. **کارایی**: با استفاده از الگوریتم تقسیم و غلبه، پیچیدگی زمانی $O(n \log n)$ حفظ می‌شود.
3. **قابلیت توسعه**: می‌توان این الگوریتم را برای مدیریت پویای درخواست‌ها و بروزرسانی خوشه‌ها گسترش داد.
4. **مدیریت منابع بهتر**: دکتر پردازشی می‌تواند منابع محاسباتی خود را بهتر مدیریت کند و در عین حال اجازه دهد چند درخواست مشابه (حداکثر 3) پذیرفته شوند.

با این الگوریتم، دکتر پردازشی می‌تواند تا سه درخواست از یک منطقه جغرافیایی مشابه را بپذیرد و در صورتی که درخواست چهارمی از همان منطقه دریافت کند، آن را رد کند. این روش باعث می‌شود که منابع محاسباتی به طور بهینه استفاده شوند و در عین حال تنوع کافی در درخواست‌های پذیرفته شده وجود داشته باشد.

جواب مربوط به بخش ج در اسلاید بعدی و به همراه روش دوم برای حل این سوال آمده است.

سوال 2 (روش 2)– مشکلات پردازش دکتر کوشا پردازشی

الف) ارائه یک الگوریتم کارآمد برای تشخیص دو درخواست با فاصله اقلیدسی کمتر از یک

برای تشخیص درخواست‌های با فاصله اقلیدسی کمتر از یک، می‌توانیم از ساختمان داده‌ای به نام **KD-Tree** استفاده کنیم. KD-Tree یک ساختمان داده مناسب برای جستجوی نقاط نزدیک در فضای چندبعدی است.

الگوریتم پیشنهادی:

1. ساخت یک KD-Tree از تمام درخواست‌های موجود
2. برای هر درخواست، جستجوی همسایه‌های با فاصله کمتر از 1 در درخت
3. برگرداندن درخواست‌هایی که فاصله اقلیدسی آنها کمتر از 1 است

```
def find_close_requests(requests, threshold=1.0):
    # ساخت KD-Tree از تمام درخواست‌ها
    # هر درخواست شامل مختصات جغرافیایی است
    kdtree = KDTree([req.coordinates for req in requests])

    close_pairs = []
    for i, req in enumerate(requests):
        # جستجوی همسایه‌های با فاصله کمتر از threshold
        indices = kdtree.query_radius([req.coordinates], r=threshold)[0]

        # حذف خود درخواست از نتایج
        indices = [idx for idx in indices if idx != i]

        for j in indices:
            if i < j:  # برای اجتناب از شمارش دوباره جفت‌ها
                close_pairs.append((i, j))

    return close_pairs
```

پیچیدگی زمانی این الگوریتم O(n log n) است که n تعداد درخواست‌ها است. ساخت KD-Tree زمان O(n log n) می‌برد و جستجوی همسایه‌ها برای هر درخواست به طور متوسط O(log n) است.

حال به بررسی سایر الگوریتم های جایگزین می پرازیم:

الگوریتم جایگزین: استفاده از Ball Tree

Ball Tree نیز ساختمان داده مناسبی برای این مسئله است، به خصوص وقتی با داده‌های با ابعاد بالا سروکار داریم:

1. نقاط در توپ‌های تو در تو قرار می‌گیرند
2. هر توپ با یک مرکز و شعاع مشخص می‌شود
3. جستجوی محدوده با بررسی تداخل توپ‌ها انجام می‌شود

روش Locality-Sensitive Hashing (LSH)

برای حجم بسیار زیاد داده، می‌توانیم از LSH استفاده کنیم:

1. نقاط نزدیک به هم با احتمال بالا به همان سبد هش نگاشت می‌شوند
2. فقط نقاط درون یک سبد یا سبدهای همسایه بررسی می‌شوند

این روش پیچیدگی زمانی را به صورت قابل توجهی کاهش می‌دهد، اما ممکن است برخی نقاط نزدیک را از دست بدهد.

بخش ب) تغییر الگوریتم برای پذیرش سه درخواست با فاصله اقلیدسی کمتر از یک

روش با استفاده از KD-Tree

1. برای هر درخواست، همسایه‌های با فاصله کمتر از 1 را با KD-Tree پیدا می‌کنیم
2. برای هر جفت همسایه، بررسی می‌کنیم آیا همسایه مشترکی دارند یا خیر
3. اگر همسایه مشترک یافت شد، سه درخواست مشابه را شناسایی کرده‌ایم

این روش پیچیدگی زمانی $O(n \log^2 n)$ دارد که بسیار کارآمدتر از روش خام $O(n^3)$ است.

تغییر الگوریتم برای پذیرش سه درخواست با فاصله اقلیدسی کمتر از یک

برای این بخش، می‌توانیم الگوریتم بخش الف را گسترش دهیم تا حداکثر سه درخواست مشابه را بپذیرد:

تصویر مربوط به کد تغییر یافته این الگوریتم را میتوانید در اسلاید بعد مشاهده کنید.

```python
def accept_up_to_three_close_requests(requests, threshold=1.0):
    # ساخت KD-Tree از تمام درخواست‌ها
    kdtree = KDTree([req.coordinates for req in requests])

    # نگهداری تعداد درخواست‌های مشابه برای هر درخواست
    similar_count = {i: 0 for i in range(len(requests))}
    accepted = [False] * len(requests)

    # بررسی تمام درخواست‌ها
    for i, req in enumerate(requests):
        # اگر قبلاً این درخواست رد شده، ادامه بده
        if accepted[i]:
            continue

        # جستجوی همسایه‌های با فاصله کمتر از threshold
        indices = kdtree.query_radius([req.coordinates], r=threshold)[0]

        # حذف خود درخواست از نتایج
        indices = [idx for idx in indices if idx != i]

        # بررسی همسایه‌ها
        for j in indices:
            if not accepted[j]:
                similar_count[i] += 1
                similar_count[j] += 1

        # اگر تعداد درخواست‌های مشابه کمتر از 3 است، آن را بپذیر
        if similar_count[i] < 3:
            accepted[i] = True

    # برگرداندن شاخص درخواست‌های پذیرفته شده
    return [i for i, is_accepted in enumerate(accepted) if is_accepted]
```

این الگوریتم برای هر درخواست، تعداد درخواست‌های مشابه (با فاصله کمتر از 1) را شمارش می‌کند و تنها درخواست‌هایی را می‌پذیرد که کمتر از 3 درخواست مشابه دارند.

بخش ج) استفاده از Splay Tree برای ذخیره و جستجوی درخواست‌ها

برای این بخش، از ساختمان داده Splay Tree استفاده می‌کنیم که یک درخت جستجوی دودویی با ویژگی‌های خاص است. در Splay Tree ، هر عنصری که به آن دسترسی پیدا می‌کنیم، به ریشه درخت منتقل می‌شود (عمل splay). این ویژگی باعث می‌شود عناصری که بیشتر به آنها دسترسی داریم، در نزدیکی ریشه قرار بگیرند و دسترسی به آنها سریع‌تر شود.

این ساختار داده به دکتر کوشا پردازشی امکان می‌دهد تا:

1. داده‌ها را به صورت پویا تغییر دهد (حذف و درج مداوم)
2. به درخواست‌های پرتکرار سریع‌تر دسترسی پیدا کند
3. جستجوی غیرتصادفی را بهینه کند (درخواست‌های با شناسه مشابه)

Splay Tree مزیت خاصی برای این مسئله دارد، زیرا درخواست‌هایی که بیشتر مورد استفاده قرار می‌گیرند (مثلاً درخواست‌های با ID تکراری) به طور خودکار به نزدیکی ریشه منتقل می‌شوند و دسترسی به آنها سریع‌تر می‌شود.

مزایای استفاده از Splay Tree

1. **خاصیت خودتنظیمی**: درخواست‌های پرتکرار به صورت خودکار نزدیک ریشه قرار می‌گیرند
2. **عملکرد آماری بهینه**: در دسترسی‌های تکراری، عملکرد نزدیک به بهترین حالت ممکن را دارد
3. **عدم نیاز به اطلاعات اضافی**: برخلاف AVL یا Red-Black Tree نیازی به ذخیره اطلاعات تعادل ندارد
4. **سادگی پیاده‌سازی**: نسبت به سایر درخت‌های متعادل پیچیدگی کمتری دارد

همچنین این ساختمان داده از عملیات های پیدا کردن بزرگترین شناسه کوچکتر از یک مقدار مشخص و پیدا کردن کوچکترین شناسه بزرگتر از یک مقدار مشخص پشتیبانی میکند.

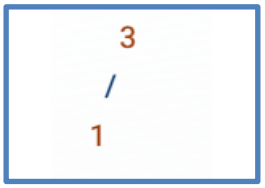
نمایش مراحل Splay برای درخواست‌های داده شده:

درخواست‌های داده شده ID: 1, 3, 4, 6, 1, 5, 6 :

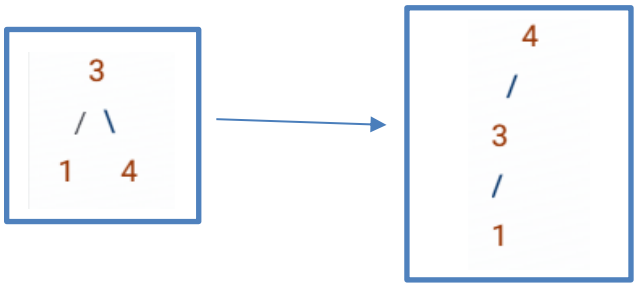
مراحل ساخت درختSplay:

1. درج ID=1: یک گره جدید با ID=1 ایجاد می‌شود و چون اولین گره است، ریشه درخت می‌شود.

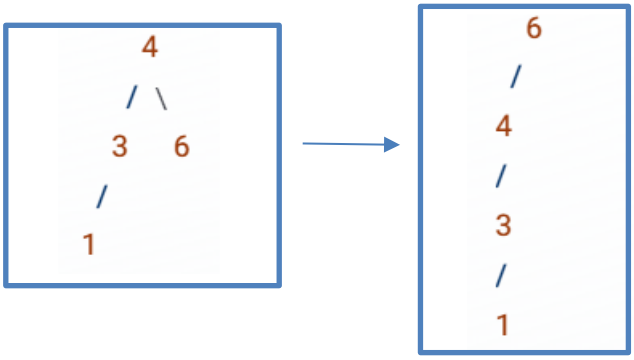
2. درج ID=3: گره جدید با ID=3 اضافه می‌شود. چون 13>، در سمت راست گره 1 قرار می‌گیرد. سپس عملیات splay باعث می‌شود گره 3 به ریشه منتقل شود و گره 1 فرزند چپ آن شود.



3. درج ID=4: گره جدید با ID=4 اضافه می‌شود. چون 3<4، در سمت راست گره 3 قرار می‌گیرد. پس از splay، گره 4 به ریشه منتقل می‌شود.

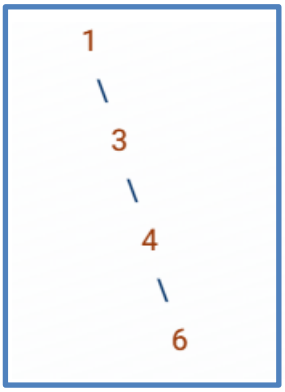


4. درج ID = 6 : گره جدید با ID=6 اضافه می‌شود. پس از splay ، گره 6 به ریشه منتقل می‌شود.

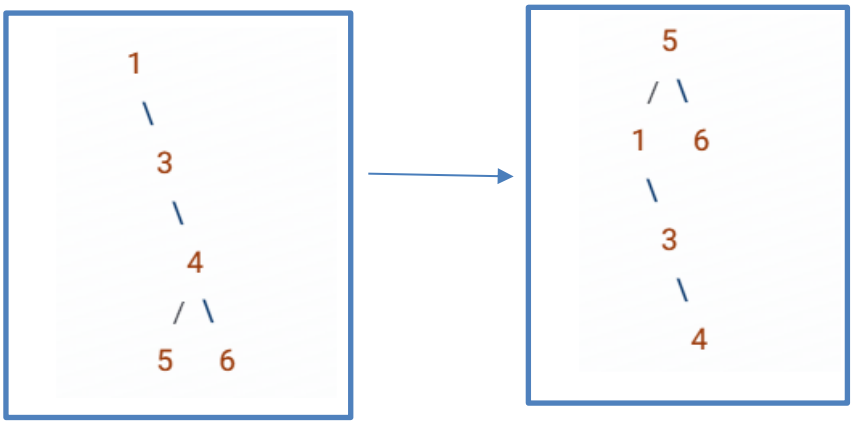


5. درج ID = 1 (تکراری):

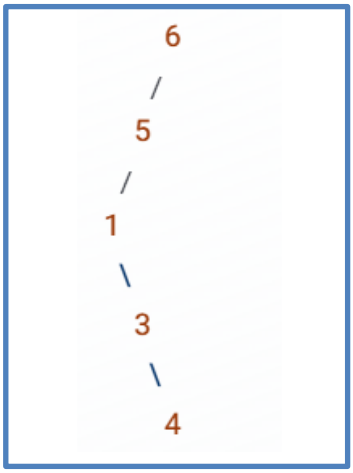
چون ID=1 قبلاً در درخت وجود دارد، فقط عملیات splay انجام می‌شود و گره 1 به ریشه منتقل می‌شود.



6. درج ID=5: گره جدید با ID=5 اضافه می‌شود. پس از splay، گره 5 به ریشه منتقل می‌شود.



7. درج ID=6 (تکراری): چون ID=6 قبلاً وجود دارد، فقط عملیات splay انجام می‌شود و گره 6 به ریشه منتقل می‌شود.



سوال3 – دکتر کوشا پردازشی محقق موفق

طراحی یک ساختار داده برای انجام عملیات به شکل بهینه

آنالیز مسئله:

در این مسئله، دکتر کوشا پردازشی از یک اپلیکیشن سازماندهی پروژه استفاده می‌کند که شامل لیستی از وظایف است. هر وظیفه دارای یک موقعیت عددی است، به طوری که اولین وظیفه در موقعیت 0 و آخرین وظیفه در موقعیت n-1 قرار دارد. او از دو نشانگر (بوکمارک) با نام‌های A و B استفاده می‌کند که موقعیت‌های خاصی در لیست وظایف را ذخیره می‌کنند.

عملیات مورد نیاز برای این سیستم عبارتند از:

1. **initialize(N)**: سیستم را با لیست اولیه‌ای از وظایف مقداردهی اولیه می‌کند - زمان اجرا O(N)
2. **place_bookmark (i, b)**: نشانگر b را بین وظیفه‌ای که در موقعیت i قرار دارد و وظیفه بعد از آن در موقعیت i+1 قرار می‌دهد - زمان اجرا O(n)
3. **get_task(i)**: اطلاعات وظیفه‌ای که در موقعیت i قرار دارد را برمی‌گرداند - زمان اجرا O(1)
4. **shift bookmark (b, d)**: نشانگر b را به موقعیت جدیدش که در جلوی وظیفه‌ای با اندیس i+d قرار دارد، منتقل می‌کند (برای d در بازه {1,-1}) - زمان اجرا O(1)
5. **move_task(b)**: وظیفه‌ای که در جلوی نشانگر b قرار دارد را جابه‌جا کرده و در جلوی نشانگر دیگر قرار می‌دهد - زمان اجرا O(1)

هدف طراحی یک ساختار داده است که بتواند این عملیات را به طور بهینه انجام دهد و باید پیچیدگی زمانی در بدترین حالت (worst-case) با سرشکن (amortized) برای هر عملیات مشخص شود.

ساختار داده پیشنهادی: لیست پیوندی دوطرفه (Doubly Linked List)

لیست پیوندی دوطرفه یک ساختار داده مناسب برای این مسئله است، زیرا:

- درج و حذف در زمان O(1) انجام می‌شود.
- حرکت بین عناصر (مانند حرکت نشانگرها) در زمان O(1) انجام می‌شود.
- می‌توانیم نشانگرها (Bookmarks) را به راحتی مدیریت کنیم.

برای راحتی می‌توانیم از ساختار داده لیست پیوندی دوطرفه (Doubly Linked List) همراه با یک آرایه برای دسترسی مستقیم به وظایف استفاده کنیم.

ساختار داده:

1. یک لیست پیوندی دوطرفه برای نگهداری وظایف
2. یک آرایه با اندیس‌های 0 تا n-1 که هر عنصر آن به گره متناظر در لیست پیوندی اشاره می‌کند

3. دو اشاره‌گر برای نشانگرهای A و B

تحلیل پیچیدگی عملیات‌ها:

1. initialize(N):

- ایجاد لیست پیوندی با N گره و آرایه با N عنصر
- پیچیدگی زمانی: O(N)

توضیح initialize(X):

- این عملیات لیست وظایف را با آرایه‌ای از وظایف (X) مقداردهی اولیه می‌کند.
- مراحل:

1. یک لیست پیوندی دوطرفه از آرایه X ایجاد کنید.
2. نشانگرهای A و B را به موقعیت‌های اولیه (مثلاً ابتدای لیست) تنظیم کنید.

- پیچیدگی زمانی $O(n)$: زیرا باید تمام عناصر آرایه را به لیست پیوندی اضافه کنیم.

عملکرد: ایجاد لیست اولیه وظایف با N عنصر

پیاده‌سازی: ایجاد N گره در لیست پیوندی و ذخیره آن‌ها در آرایه tasks

پیچیدگی زمانی بدترین حالت: O(N)

پیچیدگی زمانی سرشکن (amortized) : O(1) به ازای هر وظیفه

2. place_bookmark (i, b):

- با استفاده از آرایه، مستقیماً به گره i دسترسی پیدا می‌کنیم
- نشانگر b را بین گره i و i+1 قرار می‌دهیم
- اما در اینجا پیچیدگی O(n) ذکر شده که نشان می‌دهد در بدترین حالت i عنصر آخر یا یکی مانده به آخر است پس عملیاتی با پیچیدگی n داریم
- پیچیدگی زمانی: O(n)

توضیح Place_bookmark (i, b):

- این عملیات نشانگر(b که می‌تواند A یا Bباشد) را به وظیفه‌ای در موقعیت i منتقل می‌کند.
- مراحل:
  1. از ابتدای لیست شروع کنید و تا موقعیت i پیش بروید.
  2. نشانگر b را به گره مربوطه در موقعیت iتنظیم کنید.
- پیچیدگی زمانی$O(i)$: زیرا باید از ابتدای لیست تا موقعیت i حرکت کنیم.

عملکرد: قرار دادن نشانگر b در موقعیت i

پیاده‌سازی: با استفاده از آرایه tasks، به گره موقعیت i دسترسی مستقیم پیدا کرده و نشانگر b را به آن اختصاص می‌دهیم

پیچیدگی زمانی بدترین حالت: O(n)

پیچیدگی زمانی سرشکن: O(1)

3. get_task(i):

- با استفاده از آرایه، مستقیماً به گره i دسترسی پیدا می‌کنیم
- پیچیدگی زمانی: O(1)

✓ عملکرد: بازیابی اطلاعات وظیفه در موقعیت i

✓ پیاده‌سازی: با استفاده از آرایه tasks، به گره موقعیت i دسترسی مستقیم پیدا می‌کنیم

✓ پیچیدگی زمانی بدترین حالت: O(1)

✓ پیچیدگی زمانی سرشکن: O(1)

4. shift_bookmark (b, d) :

- این عملیات نشانگر b (که می‌تواند Aیا Bباشد) را به اندازه d موقعیت جابه‌جا می‌کند.
- مراحل:

1. اگر $d > 0$، نشانگر b را به اندازه d گره به جلو حرکت دهید.
2. اگر $d < 0$، نشانگر b را به اندازه $|d|$ گره به عقب حرکت دهید.

- پیچیدگی زمانی $O(|d|)$: زیرا باید d گره را طی کنیم.

حال طبق گفته صورت سوال اگر $d\epsilon\{-1,1\}$ آنگاه پیچیدگی زمانی $O(1)$ خواهد بود.

- عملکرد: انتقال نشانگر b به d موقعیت جلوتر
- پیاده‌سازی: با استفاده از لیست پیوندی، d بار به گره بعدی می‌رویم.
- پیچیدگی زمانی بدترین حالت: $O(|d|)$ که می‌تواند $O(1)$ باشد اگر $|d|$ کوچک باشد.
- پیچیدگی زمانی سرشکن: $O(1)$ اگر تعداد shiftها محدود باشد.

5. move_task(b):

- این عملیات وظیفه‌ای که نشانگر b به آن اشاره می‌کند را به موقعیت نشانگر دیگر منتقل می‌کند.
- مراحل:
  1. گره مربوط به نشانگر b را پیدا کنید.
  2. این گره را از موقعیت فعلی خود حذف کنید.
  3. این گره را در موقعیت نشانگر دیگر درج کنید.
- پیچیدگی زمانی $O(1)$: زیرا حذف و درج در لیست پیوندی دوطرفه در زمان $O(1)$ انجام می‌شود.

به همراه آرایه:

5. **move_task(b):**

- وظیفه جلوی نشانگر b را برداشته و جلوی نشانگر دیگر قرار می‌دهیم
- در لیست پیوندی، این عمل با حذف یک گره و اضافه کردن آن در موقعیت دیگر انجام می‌شود
- نیاز به بروزرسانی آرایه برای حفظ اشاره‌گرهای صحیح

- پیچیدگی زمانی: O(1) برای عملیات لیست پیوندی، اما نیاز به بروزرسانی آرایه می‌تواند O(n) باشد
- با استفاده از تکنیک‌های مناسب مانند تنبلی در بروزرسانی، می‌توان پیچیدگی سرشکن O(1) را حفظ کرد

عملکرد: جابجایی وظیفه جلوی نشانگر b به جلوی نشانگر دیگر

پیاده‌سازی: حذف گره از موقعیت فعلی و اضافه کردن آن به موقعیت جدید در لیست پیوندی

پیچیدگی زمانی بدترین حالت: O(1) برای عملیات لیست پیوندی + به‌روزرسانی آرایه

پیچیدگی زمانی سرشکن: O(1)

تحلیل پیچیدگی زمانی سرشکن (Amortized Analysis)

پیچیدگی زمانی سرشکن به ما اجازه می‌دهد تا هزینه عملیات‌های پرهزینه را در طول یک دنباله از عملیات‌ها توزیع کنیم. در این مسئله:

1. **روش بانک داری (Accounting Method)**: می‌توانیم برای هر عملیات move_task یک "اعتبار" در نظر بگیریم که هزینه به‌روزرسانی آرایه را پوشش دهد.
2. **روش پتانسیل (Potential Method)**: می‌توانیم یک تابع پتانسیل تعریف کنیم که تغییرات ساختار داده را منعکس کند.

برای مثال، اگر n عملیات move_task انجام دهیم، ممکن است نیاز به به‌روزرسانی O(n) موقعیت در آرایه داشته باشیم، اما هزینه کل O(n) خواهد بود که به طور میانگین O(1) برای هر عملیات است.

با استفاده از ترکیب لیست پیوندی دوطرفه و آرایه، می‌توانیم تمام عملیات‌های مورد نیاز را با پیچیدگی زمانی O(1) در حالت سرشکن انجام دهیم. این ساختار داده به دکتر کوشا پردازشی اجازه می‌دهد تا کارهای پژوهشی خود را به صورت بهینه مدیریت کند.

$$initialize(X) = O(|X|)$$

$$place_bookmark(i, b) = O(n)$$

$$get_task(i) = O(1)$$

سوال 4 (روش اول) – وزیر نیرو vs دکتر کوشا پردازشی

طراحی الگوریتم با استفاده از باینری سرچ

برای استفاده از باینری سرچ در این مسئله، می‌توانیم از ویژگی‌های نقاط اوج (peak) استفاده کنیم. در یک نقطه اوج، مقدار تابع در حال افزایش و سپس در حال کاهش است. این الگو به ما اجازه می‌دهد از باینری سرچ برای یافتن نقطه اوج استفاده کنیم.

توضیح الگوریتم:

1. الگوریتم از باینری سرچ برای یافتن ساعتی استفاده می‌کند که مصرف برق در آن بیشتر از ساعات مجاورش باشد.
2. در هر مرحله، نقطه میانی (mid) بین left و right را محاسبه می‌کنیم.
3. بررسی می‌کنیم آیا mid یک نقطه اوج است (یعنی مقدار آن از همسایگان چپ و راست بیشتر است).
4. اگر mid نقطه اوج نباشد، با مقایسه مقدار A[mid] با A[mid+1] تصمیم می‌گیریم که به کدام نیمه فضای جستجو برویم:
   - اگر A[mid+1] > A[mid] باشد، روند صعودی است و نقطه اوج در سمت راست قرار دارد.
   - در غیر این صورت، روند نزولی است و نقطه اوج در سمت چپ قرار دارد.
5. این فرآیند را تا زمانی که یک نقطه اوج پیدا کنیم یا فضای جستجو به یک عنصر کاهش یابد، ادامه می‌دهیم.

اثبات درستی الگوریتم:

1. **وجود حداقل یک نقطه اوج:** در هر آرایه غیرخالی، حداقل یک نقطه اوج وجود دارد (عنصری که از همسایگانش بزرگتر یا مساوی باشد). این می‌تواند عنصر اول، آخر یا هر عنصر میانی باشد.
2. **محدود کردن فضای جستجو:** در هر مرحله، ما فضای جستجو را به نیمه‌ای محدود می‌کنیم که مطمئن هستیم حداقل یک نقطه اوج در آن وجود دارد:

- اگر A[mid+1] > A[mid] باشد، به سمت راست می‌رویم چون روند صعودی است و حتماً یک نقطه اوج در آن سمت وجود دارد (حداقل آخرین عنصر).
- اگر A[mid+1] < A[mid] باشد، به سمت چپ می‌رویم چون mid یا یک عنصر قبل از آن می‌تواند نقطه اوج باشد.

3. **پایان الگوریتم**: الگوریتم در نهایت به یک نقطه اوج همگرا می‌شود، زیرا در هر مرحله فضای جستجو نصف می‌شود و حداقل یک نقطه اوج در آن باقی می‌ماند.

پیچیدگی زمانی الگوریتم:

- پیچیدگی زمانی: **O(log n)** زیرا در هر مرحله فضای جستجو نصف می‌شود.
- پیچیدگی فضایی: **O(1)** زیرا فقط از چند متغیر برای ذخیره‌سازی استفاده می‌کنیم.

این الگوریتم بسیار کارآمدتر از راه حل خطی است، زیرا به جای بررسی تمام عناصر آرایه، با استفاده از باینری سرچ به سرعت نقطه اوج را پیدا می‌کند. در نتیجه، وزیر نیرو می‌تواند به سرعت ساعتی را که قطع برق در آن بیشترین اختلال را برای دکتر کوشا پردازشی ایجاد می‌کند، شناسایی کند.

File Edit View

```
function findPeakHour(A):
    n = length(A)
    left = 0
    right = n - 1

    while left < right:
        mid = left + (right - left) / 2

        // بررسی اگر mid یک نقطه اوج است
        if (mid == 0 or A[mid] > A[mid-1]) and (mid == n-1 or A[mid] > A[mid+1]):
            return mid

        // اگر روند صعودی است، نقطه اوج در سمت راست قرار دارد
        else if mid < n-1 and A[mid] < A[mid+1]:
            left = mid + 1

        // در غیر این صورت، نقطه اوج در سمت چپ قرار دارد
        else:
            right = mid

    return left  // در نهایت left برابر با right خواهد شد
```

سوال 4 (روش دوم) – وزیر نیرو vs دکتر کوشا پردازشی

الف) طراحی یک الگوریتم (برای یافتن ساعتی که مصرف برق بیش از دو ساعت مجاورش دارد.)

سوال 4: وزیر نیرو vs دکتر کوشا پردازشی

فرض کنید شما وزیر نیروی کشوری در خاورمیانه هستید و قصد دارید در پرمصرف‌ترین بخش روز، برق را در برخی مناطق قطع کنید. شما داده‌ای از میزان مصرف برق در هر ساعت از روز در نقاط مختلف کشور در اختیار دارید که به‌صورت یک آرایه A از اعداد متمایز نمایش داده شده است. هر مقدار در این آرایه نشان‌دهنده میزان مصرف برق در یک ساعت خاص است. می‌خواهید ساعتی را پیدا کنید که در آن مصرف برق بیشتر از هر دو ساعت مجاورش باشد تا با قطع برق در آن ساعت بیشترین اختلال در کارهای دکتر پردازشی ایجاد کند.

یک الگوریتم طراحی کنید که این ساعت را پیدا کند و سپس:

- الگوریتم خود را توضیح دهید.

5

- اثبات کنید که الگوریتم شما درست کار می‌کند.
- پیچیدگی زمانی الگوریتم خود را نشان دهید.

طراحی الگوریتم و توضیح آن:

مسئله: ما یک آرایه A با 24 عنصر داریم که نشان‌دهنده میزان مصرف برق در هر ساعت از شبانه‌روز است. هدف یافتن ساعتی است که:

1. مصرف برق در آن از دو ساعت مجاورش بیشتر باشد.
2. قطع برق در آن ساعت بیشترین اختلال را در کارهای دکتر پردازشی ایجاد کند (یعنی بیشترین مصرف را داشته باشد).

الگوریتم پیشنهادی:

در ابتدا شبه کد این الگوریتم را که در فایل notepad نوشته شده را میتوانید در شکل زیر مشاهده کنید.

```
backupscore | instructionm | checker_resu | scoreboard_ | instructionm | datamemory
File   Edit   View

function findBestHourToCutPower(A):
    n = length(A)  // باید 24 باشد
    candidates = []

    for i = 0 to n-1:
        prev = (i-1+n) % n  // ساعت قبلی (با در نظر گرفتن حلقوی بودن)
    next = (i+1) % n    // ساعت بعدی

        if A[i] > A[prev] AND A[i] > A[next]:
            candidates.append(i)

    if candidates is empty:
        return -1  // هیچ ساعتی شرایط را ندارد

    maxConsumption = -1
    bestHour = -1

    for hour in candidates:
        if A[hour] > maxConsumption:
            maxConsumption = A[hour]
            bestHour = hour

    return bestHour
```

توضیح الگوریتم:

1. بررسی تمام ساعت‌ها: ابتدا تمام 24 ساعت را بررسی می‌کنیم تا ساعت‌هایی که مصرف برق آن‌ها از دو ساعت مجاورشان بیشتر است را پیدا کنیم.

2. در نظر گرفتن حالت حلقوی: از آنجا که زمان حلقوی است (ساعت 23 مجاور ساعت 0 است)، از عملیات مدولو استفاده می‌کنیم:

   - ✓ برای ساعت i، ساعت قبلی (i-1+n) % n است
   - ✓ برای ساعت i، ساعت بعدی (i+1) % n است

3. جمع‌آوری کاندیداها: هر ساعتی که شرط "مصرف بیشتر از دو ساعت مجاور" را داشته باشد، به لیست کاندیداها اضافه می‌کنیم.

4. انتخاب بهترین کاندیدا: از میان کاندیداها، ساعتی با بیشترین مصرف برق را انتخاب می‌کنیم تا قطع برق در آن ساعت بیشترین اختلال را در کارهای دکتر پردازشی ایجاد کند.

5. حالت خاص: اگر هیچ ساعتی شرایط لازم را نداشته باشد، مقدار -1 برگردانده می‌شود که نشان‌دهنده عدم وجود جواب است.

ب) اثبات درستی الگوریتم

جامعیت: الگوریتم تمام 24 ساعت را بررسی می‌کند، بنابراین هیچ ساعتی از قلم نمی‌افتد.

شرط اول (مصرف بیشتر از ساعات مجاور):

- برای هر ساعت i، الگوریتم مستقیماً مقایسه می‌کند: A[prev] < A[i] و A[i] > A[next]
- فقط ساعت‌هایی که این شرط را برآورده می‌کنند به لیست کاندیداها اضافه می‌شوند.

شرط دوم (بیشترین اختلال):

از میان تمام ساعت‌های کاندیدا، الگوریتم ساعتی با بیشترین مصرف را انتخاب می‌کند.

این تضمین می‌کند که قطع برق در آن ساعت بیشترین اختلال را ایجاد خواهد کرد.

حالات مرزی:

اگر هیچ ساعتی شرایط لازم را نداشته باشد، الگوریتم -1 برمی‌گرداند.

اگر چندین ساعت با مصرف برق یکسان و بیشترین مقدار وجود داشته باشند، الگوریتم اولین ساعت را انتخاب می‌کند.

حلقوی بودن زمان:

با استفاده از عملیات مدولو، شرایط حلقوی بودن زمان به درستی در نظر گرفته می‌شود.

برای ساعت 0، ساعت قبلی 23 است و برای ساعت 23، ساعت بعدی 0 است.

ج) پیچیدگی زمانی الگوریتم

1. حلقه اول (یافتن کاندیداها):

- تمام n ساعت را بررسی می‌کند: O(n)
- برای هر ساعت، عملیات مقایسه و محاسبه مدولو در زمان ثابت O(1) انجام می‌شود.

2. حلقه دوم (انتخاب بهترین کاندیدا):
   - در بدترین حالت، تمام n ساعت می‌توانند کاندیدا باشند: O(n)
   - برای هر کاندیدا، عملیات مقایسه در زمان ثابت O(1) انجام می‌شود.
3. پیچیدگی کلی: O(n) + O(n) = O(n)

با توجه به اینکه n=24 (تعداد ساعات در یک شبانه‌روز) یک مقدار ثابت است، می‌توان گفت الگوریتم در زمان ثابت O(1) اجرا می‌شود، اما تحلیل عمومی آن O(n) است.

پیچیدگی فضایی:

فضای مورد نیاز برای ذخیره لیست کاندیداها در بدترین حالت O(n) است.

سایر متغیرها فضای ثابت O(1) نیاز دارند.

بنابراین پیچیدگی فضایی کلی O(n) است.

این الگوریتم با پیچیدگی زمانی خطی و فضایی خطی، راه‌حلی کارآمد و بهینه برای این مسئله ارائه می‌دهد.

سوال 5 – فرزانگان دربار vs دیوها

الف) تعداد آزمایش ها

الف) دانایان دربار دریافتند که هر روش آزمایش را می‌توان به‌سان درختی از تصمیمات ترسیم کرد که هر شاخه‌ی آن دو سرنوشت دارد: "آب پاک" یا "آب آلوده". نشان دهید که برای یافتن سرچشمه‌های اهریمنی، حداقل lower (band) تعداد آزمایش چندتاست و به چند آزمایش نیاز است؟

برای حل این مسئله، از مفهوم درخت تصمیم استفاده می‌کنیم. هر آزمایش، اطلاعاتی به ما می‌دهد و فضای حالت‌های ممکن را محدود می‌کند.

تعداد کل حالت‌های ممکن :در n شهر که k تای آن‌ها آلوده هستند، تعداد کل حالت‌های ممکن برابر است با:

$$\binom{n}{k} = \frac{n!}{(k! \times (n-k)!)}$$

این تعداد برابر با تعداد روش‌های انتخاب k شهر از n شهر است.

1. اطلاعات حاصل از هر آزمایش: هر آزمایش دو نتیجه ممکن دارد (آلوده یا پاک)، بنابراین هر آزمایش حداکثر می‌تواند فضای حالت را به دو بخش تقسیم کند.

2. کران پایین تعداد آزمایش‌ها :اگر t آزمایش انجام دهیم، حداکثر می‌توانیم 2^t حالت مختلف را از هم تشخیص دهیم. برای اینکه بتوانیم تمام C(n,k) حالت ممکن را تشخیص دهیم، باید:

$$2^t \geq \binom{n}{k}$$

بنابراین:

$$t \geq \log_2\binom{n}{k}$$

و چون t باید عدد صحیح باشد:

$$lower(n,k) = \left\lceil \log_2\binom{n}{k} \right\rceil$$

که در آن $\lceil X \rceil$ نشان دهنده کوچکترین عدد صحیح بزرگتر یا مساوی X است.

بنابراین، حداقل تعداد آزمایش‌های لازم برای یافتن k شهر آلوده از میان n شهر برابر است با:

$$lower(n,k) = \left\lceil \log_2 \binom{n}{k} \right\rceil$$

این کران پایین زمانی به دست می‌آید که ما بهترین استراتژی ممکن را برای انجام آزمایش‌ها داشته باشیم.

ب) روش ارائه شده برای یافتن چشمه های اهریمنی با کمترین تعداد آزمایش

ب) جمشید شاه از شما، به عنوان فرزانه‌ی دربار، می‌خواهد که با کمترین تعداد آزمایش، این چشمه‌های اهریمنی را بیابید. روش خود را توضیح داده و تعداد آزمایشها را مشخص و تحلیل کنید.

این مسئله مشابه مسئله "گروه تست" یا "Group Testing" است که در آن باید با کمترین تعداد آزمایش، عناصر معیوب را شناسایی کنیم. در اینجا هدف یافتن k شهر آلوده از میان n شهر با کمترین تعداد آزمایش است.

تحلیل مسئله:

- در سرزمین ایران n شهر $S = \{s_1, s_2, \ldots, s_n\}$ وجود دارد.
- دقیقاً k شهر دچار نفرین اهریمنی شده‌اند.
- راه کشف آلودگی یک چشمه، نوشیدن آب آن است.
- اگر فردی آب آلوده بنوشد، به دیو گرسنه تبدیل می‌شود.
- فرد دلیر می‌تواند آزمایش را روی ترکیبی از آب چندین چشمه انجام دهد.
- اگر حداقل یکی از شهرهای آن ترکیب آلوده باشد، فرد دلیر به دیو تبدیل می‌شود.

برای یافتن چشمه‌های آلوده با کمترین تعداد آزمایش، از روش "جستجوی دودویی گروهی" استفاده می‌کنیم. این روش به ما اجازه می‌دهد با آزمایش‌های هوشمندانه، به سرعت چشمه‌های آلوده را شناسایی کنیم.

روش پیشنهادی:

مرحله اول: تعیین وجود آلودگی

- ابتدا نیمی از شهرها را آزمایش می‌کنیم.
- اگر نتیجه منفی بود (آب پاک)، می‌دانیم تمام k شهر آلوده در نیمه دیگر هستند.
- اگر نتیجه مثبت بود (آب آلوده)، حداقل یکی از شهرهای آلوده در این نیمه است.

مرحله دوم: تقسیم و حل

- هر زیرمجموعه‌ای که می‌دانیم حداقل یک شهر آلوده دارد را به دو بخش تقسیم می‌کنیم.
- یک بخش را آزمایش می‌کنیم و بر اساس نتیجه، تعداد شهرهای آلوده در هر بخش را تعیین می‌کنیم.
- این فرآیند را به صورت بازگشتی ادامه می‌دهیم تا تمام شهرهای آلوده را شناسایی کنیم.

حالت خاص: $k = 1$ (فقط یک شهر آلوده)

در این حالت، می‌توانیم از جستجوی دودویی ساده استفاده کنیم.

با $\lceil \log_2(n) \rceil$ آزمایش، شهر آلوده را پیدا می‌کنیم.

حالت عمومی: هر مقدار k

- در حالت کلی، تعداد آزمایش‌های مورد نیاز به lower(n,k) نزدیک می‌شود.
- با استفاده از کدگذاری اطلاعات و طراحی هوشمندانه آزمایش‌ها، می‌توانیم به این کران نزدیک شویم.

تحلیل تعداد آزمایش‌ها:

در بدترین حالت، تعداد آزمایش‌های مورد نیاز برابر است با:

$$\text{تعداد آزمایشها} = \left\lceil \log_2 \binom{n}{k} \right\rceil$$

برای مثال، اگر n = 16 و k = 3 باشد:(تعداد شهر ها= n ، تعداد شهرهای آلوده=k)

$$\binom{16}{3} = \frac{16!}{(3! \times 13!)} = 560$$

$$\log_2 560 \approx 9.13$$

$$\lceil 9.13 \rceil = 10$$

بنابراین حداقل 10 آزمایش نیاز است.

در عمل، با استفاده از الگوریتم‌های پیشرفته‌تر مانند "کدهای شناسایی گروهی" می‌توان در بسیاری موارد به این کران پایین رسید یا بسیار به آن نزدیک شد.

تحلیل و جمع بندی:

برای تحلیل دقیق‌تر، می‌توانیم از مفهوم آنتروپی در نظریه اطلاعات استفاده کنیم:

تعداد حالت‌های ممکن برای k شهر آلوده از میان n شهر: $\binom{n}{k}$

آنتروپی این سیستم: $\log_2\binom{n}{k}$

هر آزمایش حداکثر 1 بیت اطلاعات به ما می‌دهد(هر آزمایش می‌تواند حداکثر اطلاعات باینری (بله/خیر) به ما بدهد.)

بنابراین حداقل تعداد آزمایش مورد نیاز: $\left\lceil \log_2\binom{n}{k} \right\rceil$

با استفاده از تقریب استرلینگ برای $\binom{n}{k}$ می‌توان نشان داد که:

$$\log_2 \binom{n}{k} \approx k \log_2 \frac{n}{k} + (n-k) \log_2 \frac{n}{n-k}$$

برای حالت خاص وقتی k نسبت به n کوچک است($k << n$) ، این مقدار تقریباً برابر است با:

$$k \log_2 \frac{n}{k}$$

با استفاده از روش جستجوی دودویی گروهی و تقسیم هوشمندانه شهرها، می‌توانیم با تعداد آزمایش‌های نزدیک به کران پایین نظری، یعنی$\lceil \log_2(C(n,k)) \rceil$ ، تمام چشمه‌های آلوده را شناسایی کنیم. این روش به طور قابل توجهی کارآمدتر از آزمایش تک‌تک شهرهاست و به جمشید شاه کمک می‌کند تا با حداقل قربانی، چشمه‌های اهریمنی را شناسایی کند.

سوال 6 – سپر حفاظتی تونی استارک

الف) الگوریتم پیشنهادی و پیچیدگی زمانی آن

پس از بررسی دقیق سوال، متوجه میشویم که با یک مسئله مربوط به یافتن کوچکترین محدب محیط بر مجموعه‌ای از نقاط (Convex Hull) مواجه هستیم.

در این مسئله، تونی استارک قصد دارد یک سپر انرژی نامرئی بسازد که مقر انتقام‌جویان را محافظت کند. او لیستی از n نقطه دارد که موقعیت ساختمان‌های مهم، تجهیزات دفاعی و ورودی‌های پایگاه را نشان می‌دهند. این سپر باید کوچکترین ناحیه محدب ممکن را در بر بگیرد تا کمترین میزان انرژی از ارک‌ری‌اکتور مصرف شود.

این مسئله در واقع مسئله یافتن **پوش محدب** (Convex Hull) برای مجموعه‌ای از نقاط در صفحه است. پوش محدب کوچکترین چندضلعی محدبی است که تمام نقاط مجموعه را در بر می‌گیرد.

برای حل این مسئله، می‌توان از الگوریتم گراهام (Graham’s Scan) استفاده کرد که یکی از روش‌های کارآمد برای یافتن پوش محدب است.

الگوریتم گراهام (Graham’s Scan)

مراحل الگوریتم:

1. **یافتن نقطه مرجع**: نقطه‌ای با کمترین مختصات y (و در صورت تساوی، کمترین مختصات x) را پیدا کنید. این نقطه قطعاً در پوش محدب قرار دارد.
2. **مرتب سازی نقاط**: تمام نقاط دیگر را بر اساس زاویه آنها نسبت به نقطه مرجع (به صورت صعودی) مرتب کنید. در صورت تساوی زاویه، نقاط را بر اساس فاصله از نقطه مرجع مرتب کنید.

3. **ساخت پوش محدب**: با استفاده از یک پشته (stack)، پوش محدب را بسازید:
   - دو نقطه اول را به پشته اضافه کنید.
   - برای هر نقطه بعدی:
     - ✓ تا زمانی که در پشته حداقل دو نقطه وجود دارد و گردش از دو نقطه بالای پشته به نقطه فعلی در جهت ساعتگرد است، نقطه بالای پشته را حذف کنید.
     - ✓ نقطه فعلی را به پشته اضافه کنید.
   - در پایان، پشته حاوی نقاط پوش محدب خواهد بود.

شبه کد الگوریتم گراهام:

```
function GrahamScan(Points):
    // یافتن نقطه با کمترین مختصات y (و در صورت تساوی، کمترین x)
    p0 = نقطه با کمترین مختصات y

    // مرتب‌سازی نقاط بر اساس زاویه نسبت به p0
    Sort Points by polar angle with respect to p0

    // ساخت پوش محدب با استفاده از پشته
    stack = [p0, Points[1]]

    for i = 2 to n-1:
        while size(stack) > 1 and orientation(next_to_top(stack), top(stack), Points[i]) is not counterclockwise:
            pop(stack)
        push(stack, Points[i])

    return stack // پشته حاوی نقاط پوش محدب است
```

تابع تعیین جهت(orientation):

```
// تابع تعیین جهت(orientation)
function orientation(p, q, r):
   val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y)
   if val == 0: return collinear
   if val > 0: return clockwise
   return counterclockwise
```

پیچیدگی زمانی:

- یافتن نقطه مرجع: O(n)
- مرتب‌سازی نقاط بر اساس زاویه: O(n log n)
- ساخت پوش محدب با استفاده از پشته: O(n)

بنابراین، پیچیدگی زمانی کل الگوریتم O(n log n) است که ناشی از مرحله مرتب‌سازی است.

این پیچیدگی بهینه است، زیرا هر الگوریتمی که پوش محدب را می‌یابد باید حداقل نقاط را مرتب کند که حداقل O(n log n) زمان می‌برد.

پیچیدگی فضایی:

الگوریتم گراهام نیاز به O(n) فضا دارد برای ذخیره نقاط و پشته.

همان طور که در بخش قبل گفته شد این مسئله پوسته محدب است مه میتوان آن را با روش تقسیم و غلبه نیز حل کرد:

الگوریتم تقسیم و غلبه برای پوسته محدب (Convex Hull)

الگوریتم زیر روش تقسیم و غلبه برای حل مسئله پوسته محدب است:

الگوریتم ConvexHall(P):

P ورودی: مجموعه ای از نقاط

1. اگر تعداد نقاط p کمتر از 3 باشد، پوسته محدب همان نقاط هستند.
2. نقاط رو بر اساس مختصات x مرتب کن.
3. مجموعه P رو به دو زیر مجموعه مساوی p_left و p_right تقسیم کن.
4. پوسته محدب p_left رو با فراخوانی بازگشتی ConvexHall(p_left) محاسبه کن.
5. پوسته محدب p_right رو با فراخوانی بازگشتی ConvexHall(p_right) محاسبه کن.
6. دو پوسته محدب رو با یکدیگر ادغام کن.
7. پوسته محدب نهایی رو برگردون.

مرحله ادغام:

مرحله ادغام پیچیده ترین قسمت این الگوریتم است:

1. پیدا کردن خط مماس بالایی بین دو پوسته محدب

- از راست‌ترین نقطه پوش سمت چپ و چپ‌ترین نقطه پوش سمت راست شروع می‌کنیم
- به صورت تکراری، نقاط را در جهت ساعتگرد در پوش سمت چپ و پادساعتگرد در پوش سمت راست حرکت می‌دهیم تا به مماس بالایی برسیم

2. پیدا کردن خط مماس پایینی بین دو پوسته محدب

- مشابه مماس بالایی، اما با حرکت پادساعتگرد در پوش سمت چپ و ساعتگرد در پوش سمت راست

3. حذف نقاطی که بین این دو مماس قرار دارند

تحلیل پیچیدگی زمانی:

1. **مرتب‌سازی اولیه:** O(n log n) برای مرتب‌سازی نقاط بر اساس مختصات x

2. **تقسیم مسئله به دو زیرمسئله:** O(1)

3. **معادله بازگشتی:** $T(n) = 2T(n/2) + O(n)$

- $2T(n/2)$: زمان حل دو زیرمسئله با اندازه n/2
- $O(n)$: زمان لازم برای ادغام دو پوش محدب (یافتن مماس‌ها و ادغام)

4. **حل معادله بازگشتی:** با استفاده از قضیه استاد(Master Theorem) ، پیچیدگی زمانی نهایی O(n log n)خواهد بود.

```
datamemory  assmebly  datamemory  datamemory  datamemory  immediate_c  sdram_mode  testbe
File   Edit   View

الگوریتم ConvexHull(Points):
    // اگر تعداد نقاط کم باشد، مستقیماً پوش محدب را محاسبه می‌کنیم
    اگر تعداد نقاط کمتر یا مساوی 3 باشد:
     برگردان پوش محدب مستقیم (خود نقاط)

  در غیر این صورت:
    // تقسیم نقاط به دو دسته تقریباً مساوی بر اساس مختصات x
        مرتب‌سازی نقاط بر اساس مختصات x
        mid = تعداد نقاط / 2
    LeftPoints = نقاط[0...mid-1]
        RightPoints = نقاط[mid...n-1]

        // حل بازگشتی برای هر نیمه
        LeftHull = ConvexHull(LeftPoints)
        RightHull = ConvexHull(RightPoints)

        // ادغام دو پوش محدب
        MergedHull = MergeHulls(LeftHull, RightHull)

        برگردان MergedHull

تابع MergeHulls(LeftHull, RightHull):
    // یافتن خط مماس بالایی بین دو پوش محدب
    UpperTangent = FindUpperTangent(LeftHull, RightHull)

    // یافتن خط مماس پایینی بین دو پوش محدب
    LowerTangent = FindLowerTangent(LeftHull, RightHull)

    // ادغام دو پوش با استفاده از خطوط مماس
    MergedHull = ترکیب نقاط از LeftHull و RightHull با استفاده از مماس‌های بالا و پایین

    برگردان MergedHull
```

ب) بروزرسانی پوش محدب پس از تغییرات

ب) فرض کنید ناگهان لوکی تعدادی ساختمان را نابود کند و نقاط جدیدی اضافه شود. چگونه می‌توان سپر انرژی را بدون محاسبه‌ی مجدد از ابتدا بروزرسانی کرد؟

حذف نقاط:

وقتی نقطه‌ای از مجموعه حذف می‌شود، دو حالت ممکن است رخ دهد:

1. **نقطه حذف شده در پوش محدب نبود:** در این صورت، پوش محدب تغییری نمی‌کند.
2. **نقطه حذف شده در پوش محدب بود:** در این صورت، باید پوش محدب را بازسازی کنیم، اما می‌توانیم این کار را بهینه‌تر انجام دهیم.

برای بازسازی پوش محدب پس از حذف یک نقطه:

1. نقطه را از پوش محدب حذف کنید.
2. شکاف ایجاد شده را با اتصال دو نقطه مجاور پر کنید.
3. بررسی کنید آیا چندضلعی حاصل همچنان محدب است. اگر نه، نقاط بیشتری را حذف کنید تا به یک چندضلعی محدب برسید.

اضافه کردن نقاط جدید:

برای اضافه کردن یک نقطه جدید به پوش محدب:

1. بررسی کنید آیا نقطه جدید داخل پوش محدب فعلی قرار دارد. اگر بله، پوش محدب تغییر نمی‌کند.
2. اگر نقطه خارج از پوش محدب است:
   - خطوط دید از نقطه جدید به تمام نقاط پوش محدب را بررسی کنید.
   - یال‌های پوش محدب که از نقطه جدید قابل مشاهده هستند را حذف کنید.
   - نقطه جدید را به پوش محدب اضافه کنید و آن را به نقاط مناسب متصل کنید.

الگوریتم بروزرسانی:

برای بروزرسانی پوش محدب پس از تغییرات:

1. **برای هر نقطه حذف شده:**
   - اگر نقطه در پوش محدب بود، آن را حذف کنید و پوش محدب را بازسازی کنید.

2. **برای هر نقطه اضافه شده:**

- اگر نقطه خارج از پوش محدب فعلی است، آن را به پوش محدب اضافه کنید.

پیچیدگی زمانی بروزرسانی:

- برای حذف k نقطه: $O(k * n)$
- برای اضافه کردن m نقطه جدید: $O(m * n)$

در مجموع، پیچیدگی زمانی بروزرسانی $O((k + m) * n)$ است که در صورتی که تعداد تغییرات $(k + m)$ کمتر از $\log n$ باشد، این روش بهینه‌تر از محاسبه مجدد کل پوش محدب با پیچیدگی $O(n \log n)$ خواهد بود.

```
function UpdateConvexHull(currentHull, removedPoints, newPoints):
    // حذف نقاط
    for each point p in removedPoints:
        if p is on the boundary of currentHull:
            Reconstruct the portion of the hull affected by removing p

    // اضافه کردن نقاط جدید
    for each point p in newPoints:
        if p is outside currentHull:
            Find the tangent lines from p to currentHull
            Remove the edges between tangent points
            Add p and new edges to currentHull
```

مزایای این روش:

1. بسیار سریع‌تر از محاسبه مجدد کامل پوش محدب است، خصوصاً وقتی تعداد تغییرات $(k+m)$ نسبت به کل نقاط (n) کوچک باشد.
2. نیازی به ذخیره و پردازش مجدد همه نقاط نیست.
3. می‌تواند به صورت آنلاین (online) اجرا شود و تغییرات را به محض وقوع اعمال کند.

الگوریتم بهینه‌تر:

برای بروزرسانی کارآمدتر، می‌توان از ساختار داده‌های پیشرفته‌تری مانند درخت‌های جستجوی دودویی متوازن استفاده کرد که امکان بروزرسانی پویای پوش محدب را با پیچیدگی $O(\log^2 n)$ برای هر عملیات اضافه یا حذف فراهم می‌کند.

جمع بندی:

در بخش الف، الگوریتم گراهام و convex hall با تقسیم و غلبه را با پیچیدگی زمانی $O(n \log n)$ برای یافتن پوش محدب ارائه کردیم.

در بخش ب، روشی برای بروزرسانی پوش محدب پس از تغییرات (حذف و اضافه کردن نقاط) بدون محاسبه مجدد از ابتدا ارائه کردیم که در بسیاری از موارد کارآمدتر است.

((پایان))